

Практическая работа 35. Добавление Compose в приложение, основанное на представлении

С самого начала Jetpack Compose был разработан с учетом совместимости с View, что означает, что Compose и система View могут совместно использовать ресурсы и работать бок о бок для отображения пользовательского интерфейса. Эта функциональность позволяет добавить Compose в существующее приложение на основе View. Это означает, что Compose и Views могут сосуществовать в вашей кодовой базе до тех пор, пока все ваше приложение не будет полностью на Compose.

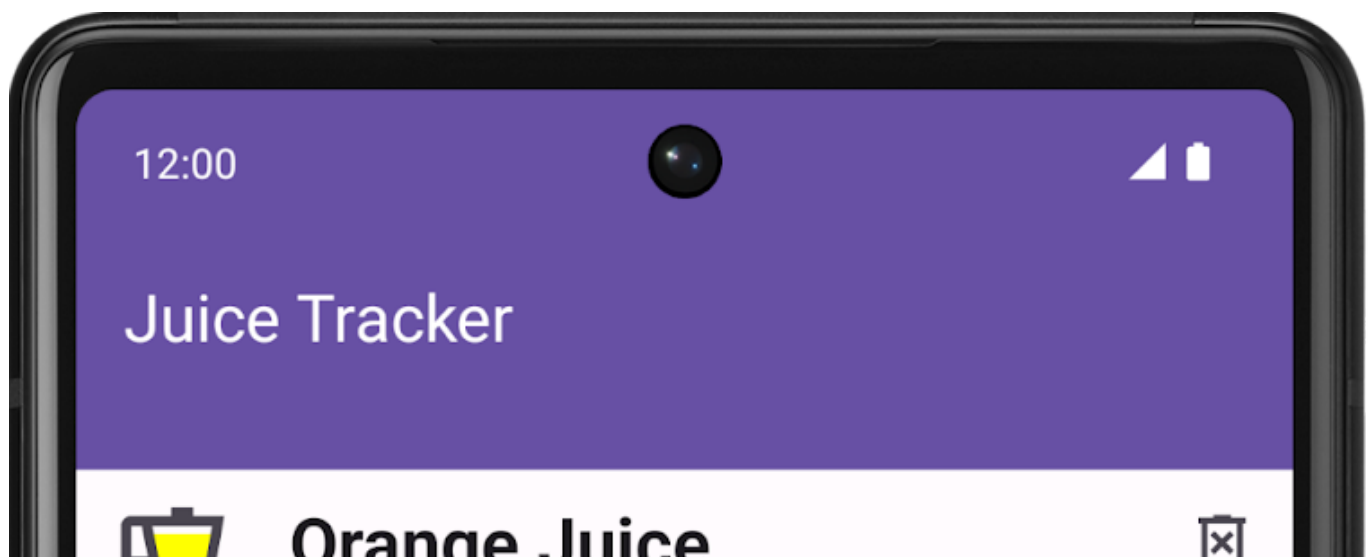
В этом уроке вы измените элемент списка на основе представлений в приложении **Juice Tracker** на Compose. При желании вы можете самостоятельно преобразовать остальные представления **Juice Tracker**.

Если у вас есть приложение с пользовательским интерфейсом на основе представлений, возможно, вы не захотите переписывать весь его пользовательский интерфейс сразу. Этот коделаб поможет вам преобразовать одно представление в пользовательском интерфейсе, основанном на представлениях, в элемент Compose.

Необходимые условия Знакомство с пользовательским интерфейсом на основе представлений. Знания о том, как создать приложение с использованием пользовательского интерфейса на основе представлений. Опыт работы с синтаксисом Kotlin, включая лямбды. Знания о том, как создавать приложения в Jetpack Compose. Что вы узнаете Как добавить Compose в существующий экран, построенный с использованием представлений Android. Как сделать предварительный просмотр Composable, добавленного в приложение, основанное на представлениях. Что вы будете создавать Вы преобразуете элемент списка на основе представлений в Compose в приложении **Juice Tracker**.

2. Обзор стартового приложения

В этом коделабе в качестве начального кода используется код решения приложения **Juice Tracker** из раздела Build an Android App with Views. Стартовое приложение уже сохраняет данные с помощью библиотеки Room persistence. Пользователь может добавлять информацию о соке в базу данных приложения, например, название, описание, цвет и рейтинг сока.





Orange Juice

Pure Valencia orange juice



Green Smotthie

Green apple, cucumber, spinach, grapes



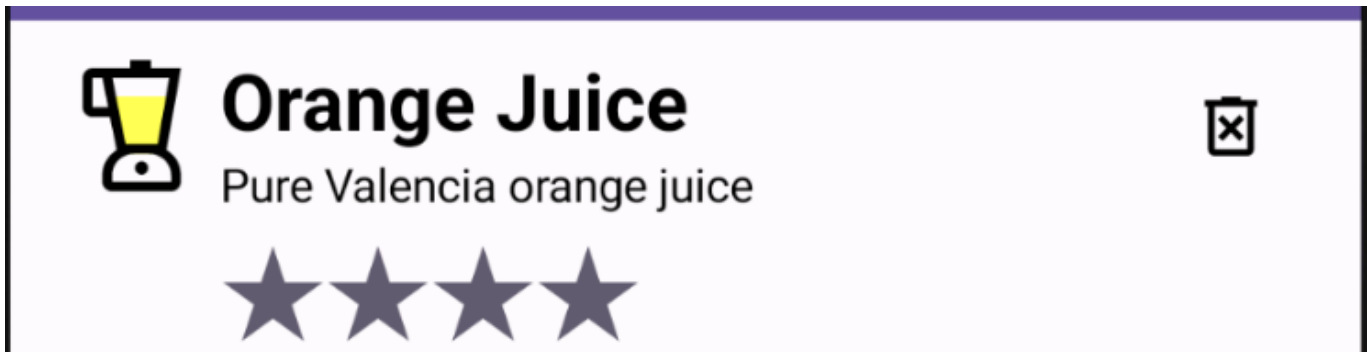
Apple Juice

Honeycrisp apple, Fuji apple





В этом руководстве вы преобразуете элемент списка на основе представления в Compose.



Загрузите стартовый код для этой кодовой лаборатории

Чтобы начать работу, загрузите стартовый код:

file_downloadDownload zip

Также вы можете клонировать репозиторий GitHub для кода:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-juice-tracker.git
$ cd basic-android-kotlin-compose-training-juice-tracker
$ git checkout views
```

Примечание: стартовый код находится в ветке views загруженного репозитория.

3. Добавьте библиотеку Jetpack Compose

Recollect, Compose и Views могут существовать вместе на одном экране; вы можете располагать некоторые элементы пользовательского интерфейса в Compose, а другие - в системе View. Например, в Compose может быть только список, а остальная часть экрана находится в системе View.

Выполните следующие шаги, чтобы добавить библиотеку Compose в приложение **Juice Tracker**.

Откройте приложение **Juice Tracker** в Android Studio. Откройте файл `build.gradle.kts` на уровне приложения. Внутри блока `buildFeatures` добавьте флаг `compose = true`.

```
buildFeatures {
    //...
    // Enable Jetpack Compose for this module
}
```

```
compose = true  
}
```

Этот флаг позволяет Android Studio работать с Compose. Вы не выполняли этот шаг в предыдущих коделабах, потому что Android Studio автоматически генерирует этот код при создании нового проекта шаблона Android Studio Compose.

Ниже buildFeatures добавьте блок composeOptions. Внутри блока установите kotlinCompilerExtensionVersion в значение «1.5.1», чтобы задать версию компилятора Kotlin.

```
composeOptions {  
    kotlinCompilerExtensionVersion = "1.5.1"  
}
```

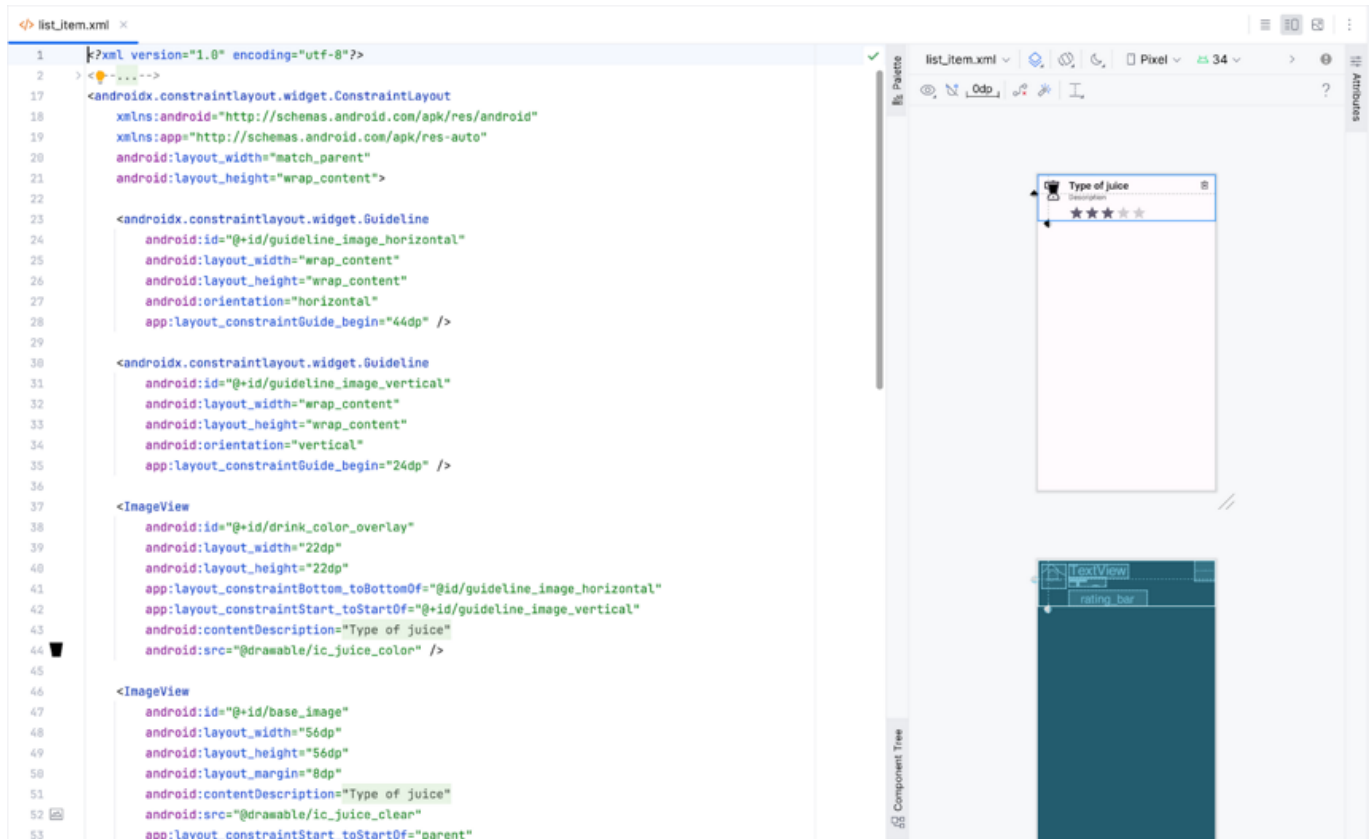
В разделе зависимостей добавьте зависимости Compose. Чтобы добавить Compose в приложение на основе View, вам понадобятся следующие зависимости. Эти зависимости помогают интегрировать Compose с Activity, добавляют библиотеку компонентов дизайна Compose, поддерживают тематическое оформление Compose Jetpack и предоставляют инструменты для лучшей поддержки IDE.

```
dependencies {  
    implementation(platform("androidx.compose:compose-bom:2023.06.01"))  
    // other dependencies  
    // Compose  
    implementation("androidx.activity:activity-compose:1.7.2")  
    implementation("androidx.compose.material3:material3")  
    implementation("com.google.accompanist:accompanist-themeadapter-material3:0.28.0")  
  
    debugImplementation("androidx.compose.ui:ui-tooling")  
}
```

Добавить ComposeView ComposeView - это представление Android, в котором может размещаться содержимое Jetpack Compose UI. Используйте setContent, чтобы задать функцию композиции контента для представления.

Откройте layout/list_item.xml и просмотрите предварительный просмотр на вкладке Split.

К концу этого коделаба вы замените это представление на композитное.



В `JuiceListAdapter.kt` удалите `ListItemBinding` из всех классов. В классе `JuiceViewHolder` замените `binding.root` на `composeView`.

```

import androidx.compose.ui.platform.ComposeView

class JuiceViewHolder(
    private val onEdit: (Juice) -> Unit,
    private val onDelete: (Juice) -> Unit
): RecyclerView.ViewHolder(composeView)
  
```

В папке `onCreateViewHolder()` обновите функцию `return()`, чтобы она соответствовала следующему коду:

```

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
JuiceViewHolder {
    return JuiceViewHolder(
        ComposeView(parent.context),
        onEdit,
        onDelete
    )
}
  
```

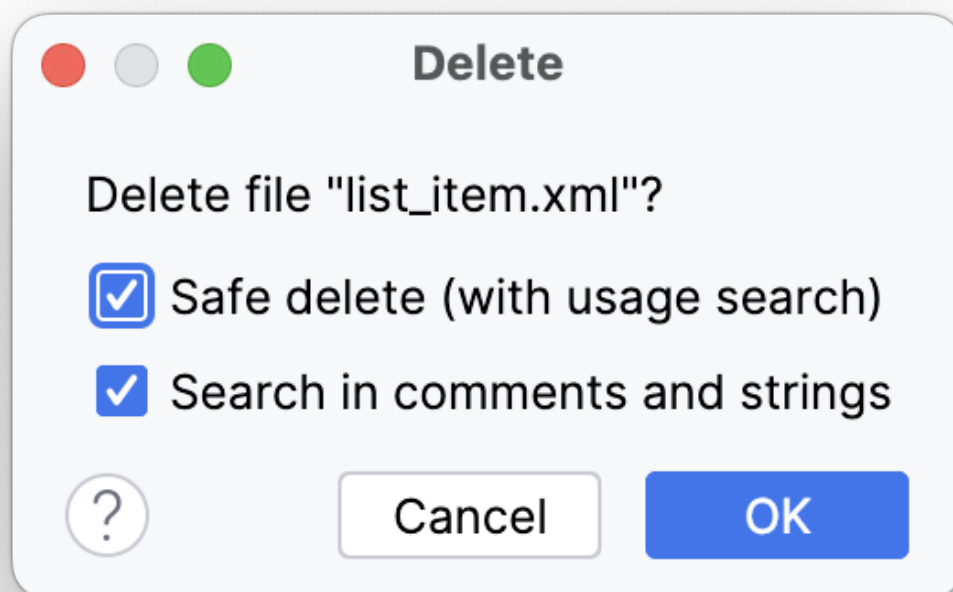
В классе `JuiceViewHolder` удалите все приватные переменные и удалите весь код из функции `bind()`. Теперь ваш класс `JuiceViewHolder` выглядит следующим образом:

```
class JuiceListViewHolder(  
    private val onEdit: (Juice) -> Unit,  
    private val onDelete: (Juice) -> Unit  
) : RecyclerView.ViewHolder(composeView) {  
  
    fun bind(juice: Juice) {  
  
    }  
}
```

На этом этапе вы можете удалить импорты `com.example.juicetracker.databinding.ListItemBinding` и `android.view.LayoutInflater`.

```
// Delete  
import com.example.juicetracker.databinding.ListItemBinding  
import android.view.LayoutInflater
```

Удалите файл `layout/list_item.xml`. Выберите OK в диалоговом окне удаления.



4. Добавьте композитную функцию

Далее вы создаете композитную функцию, которая испускает элемент списка. Composable принимает Juice и две функции обратного вызова для редактирования и удаления элемента списка.

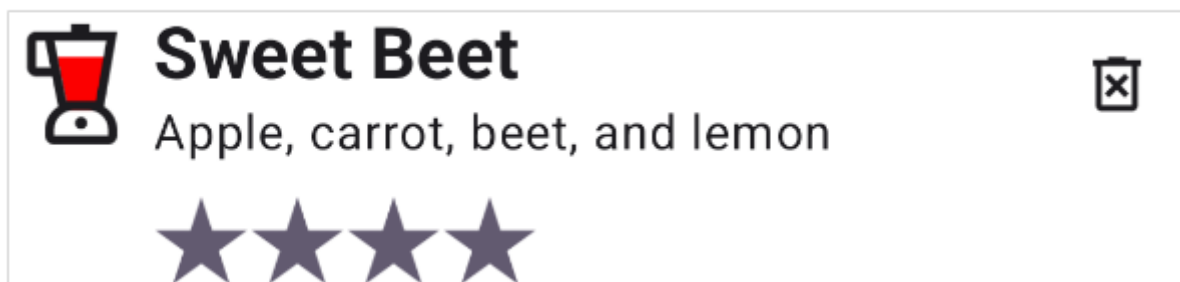
В файле JuiceListAdapter.kt после определения класса JuiceListAdapter создайте композитную функцию ListItem(). Пусть функция ListItem() принимает объект Juice и обратный лямбда-вызов для удаления.

```
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier

@Composable
fun ListItem(
    input: Juice,
    onDelete: (Juice) -> Unit,
    modifier: Modifier = Modifier
) {
}
```

Посмотрите на предварительный просмотр элемента списка, который вы хотите создать. Обратите внимание, что у него есть значок сока, сведения о соке и значок кнопки удаления. Эти компоненты вы будете реализовывать в ближайшее время.

PreviewListItem



Создание композитной функции иконки сока В файле JuiceListAdapter.kt после композитной функции ListItem() создайте еще одну композитную функцию JuiceIcon(), которая принимает цвет и модификатор.

```
@Composable
fun JuiceIcon(color: String, modifier: Modifier = Modifier) {
}
```

Внутри функции JuiceIcon() добавьте переменные для цвета и описания содержимого, как показано в следующем коде:

```
@Composable
fun JuiceIcon(color: String, modifier: Modifier = Modifier) {
    val colorLabelMap = JuiceColor.values().associateBy { stringResource(it.label) }
}
```

```

    val selectedColor = colorLabelMap[color]?.let { Color(it.color) }
    val juiceIconContentDescription = stringResource(R.string.juice_color, color)

}

```

Используя переменные `colorLabelMap` и `selectedColor`, вы получите цветовой ресурс, связанный с выбором пользователя.

Добавьте макет `Box` для отображения двух иконок `ic_juice_color` и `ic_juice_clear` друг над другом. Иконка `ic_juice_color` имеет оттенок и выровнена по центру.

```

import androidx.compose.foundation.layout.Box

Box(
    modifier.semantics {
        contentDescription = juiceIconContentDescription
    }
) {
    Icon(
        painter = painterResource(R.drawable.ic_juice_color),
        contentDescription = null,
        tint = selectedColor ?: Color.Red,
        modifier = Modifier.align(Alignment.Center)
    )
    Icon(painter = painterResource(R.drawable.ic_juice_clear), contentDescription =
null)
}

```

Поскольку вы знакомы с композитной реализацией, подробности о том, как она реализована, не приводятся.

Добавьте функцию для предварительного просмотра `JuiceIcon()`. Передайте цвет как `Yellow`.

```

import androidx.compose.ui.tooling.preview.Preview

@Preview
@Composable
fun PreviewJuiceIcon() {
    JuiceIcon("Yellow")
}

```

Создание композитных функций для отображения подробностей о соке в `JuiceListAdapter.kt` нужно добавить еще одну композитную функцию для отображения подробностей о соке. Также вам понадобится макет колонки для отображения двух композитов `Text` для названия и описания, а также индикатора рейтинга. Для этого выполните следующие действия:

Добавьте композитную функцию `JuiceDetails()`, которая принимает объект `Juice` и модификатор, а также композитный текст для названия сока и композитный текст для описания сока, как показано в следующем коде:

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.ui.text.font.FontWeight

@Composable
fun JuiceDetails(juice: Juice, modifier: Modifier = Modifier) {
    Column(modifier, verticalArrangement = Arrangement.Top) {
        Text(
            text = juice.name,
            style = MaterialTheme.typography.h5.copy(fontWeight = FontWeight.Bold),
        )
        Text(juice.description)
        RatingDisplay(rating = juice.rating, modifier = Modifier.padding(top =
8.dp))
    }
}
```

Чтобы устранить ошибку с неразрешенной ссылкой, создайте составную функцию `RatingDisplay()`.

```
@Composable
fun JuiceDetails(juice: Juice, modifier: Modifier = Modifier) {
    Column(modifier, verticalArrangement = Arrangement.Top) { this: ColumnScope
        Text(
            text = juice.name,
            style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold),
        )
        Text(juice.description)
        RatingDisplay(rating = juice.rating, modifier = Modifier.padding(top = 8.dp))
    }
```

- ❗ Create @Composable function 'RatingDisplay'
- ❗ Create class 'RatingDisplay'
- ❗ Create function 'RatingDisplay'
- ❗ Rename reference
- ❗ Create extension function 'ColumnScope.RatingDisplay'

Create new scratch file from selection
Put arguments on separate lines
Surround with widget
Adjust code style settings
Do not show return expression hints

Press F1 to toggle preview

В системе View есть элемент RatingBar для отображения следующей строки рейтинга. В Compose нет композитной панели рейтинга, поэтому вам нужно реализовать этот элемент с нуля.

Определите функцию RatingDisplay() для отображения звезд в соответствии с рейтингом. Эта композитная функция отображает количество звезд в зависимости от рейтинга.

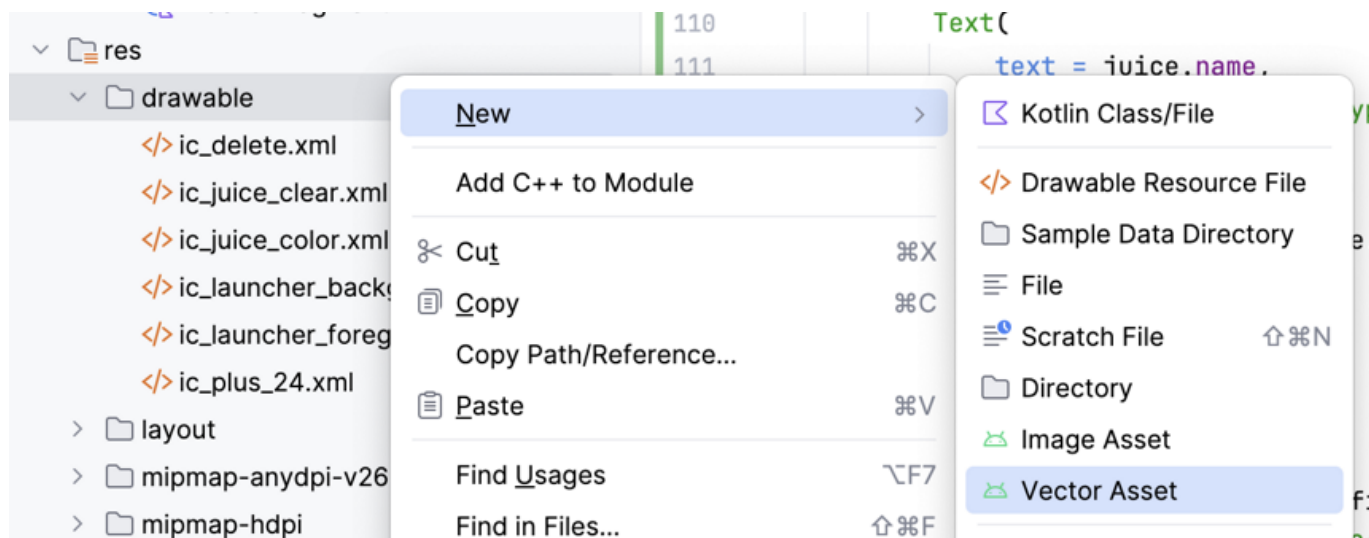


```
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.pluralStringResource

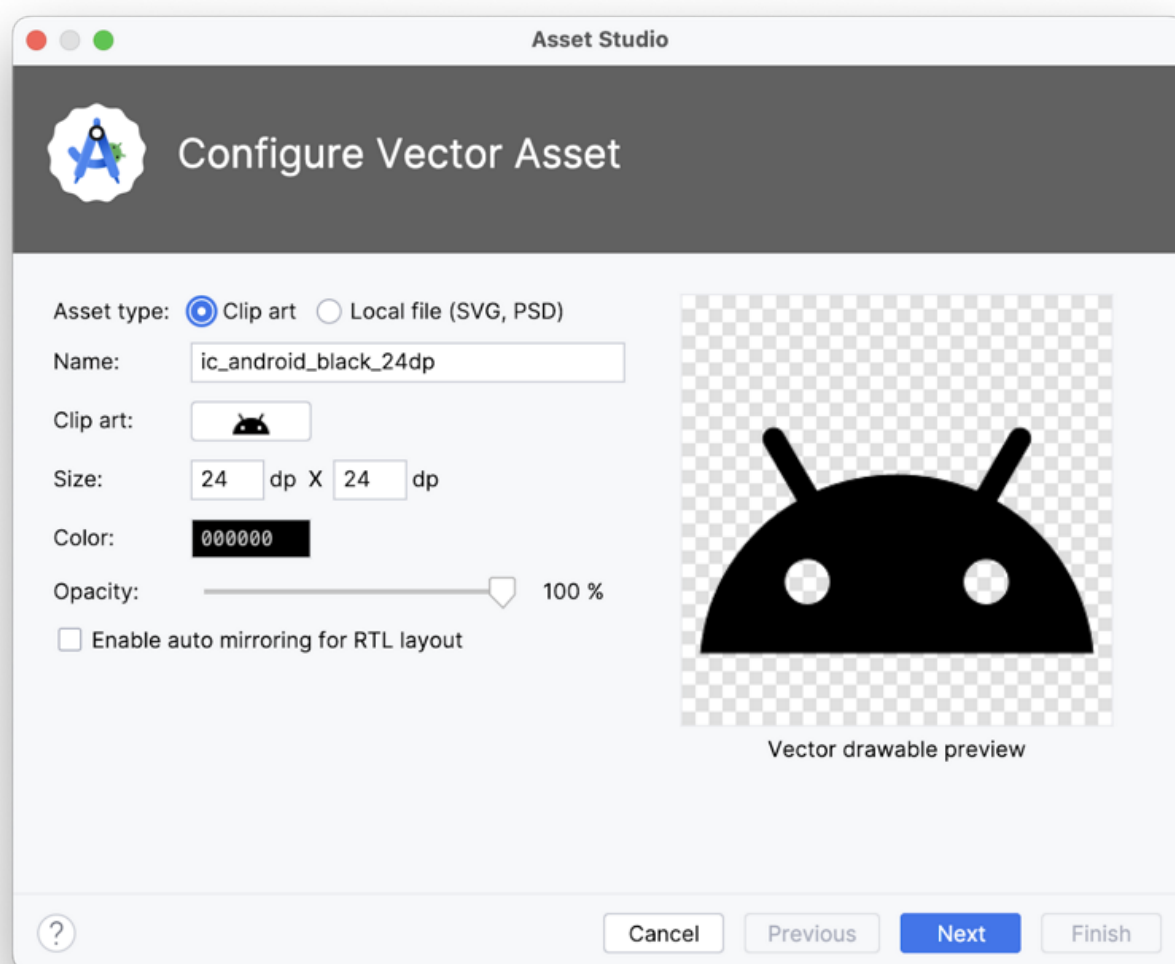
@Composable
fun RatingDisplay(rating: Int, modifier: Modifier = Modifier) {
    val displayDescription = pluralStringResource(R.plurals.number_of_stars, count
= rating)
    Row(
        // Content description is added here to support accessibility
        modifier.semantics {
            contentDescription = displayDescription
        }
    ) {
        repeat(rating) {
            // Star [contentDescription] is null as the image is for illustrative
purpose
            Image(
                modifier = Modifier.size(32.dp),
                painter = painterResource(R.drawable.star),
                contentDescription = null
            )
        }
    }
}
```

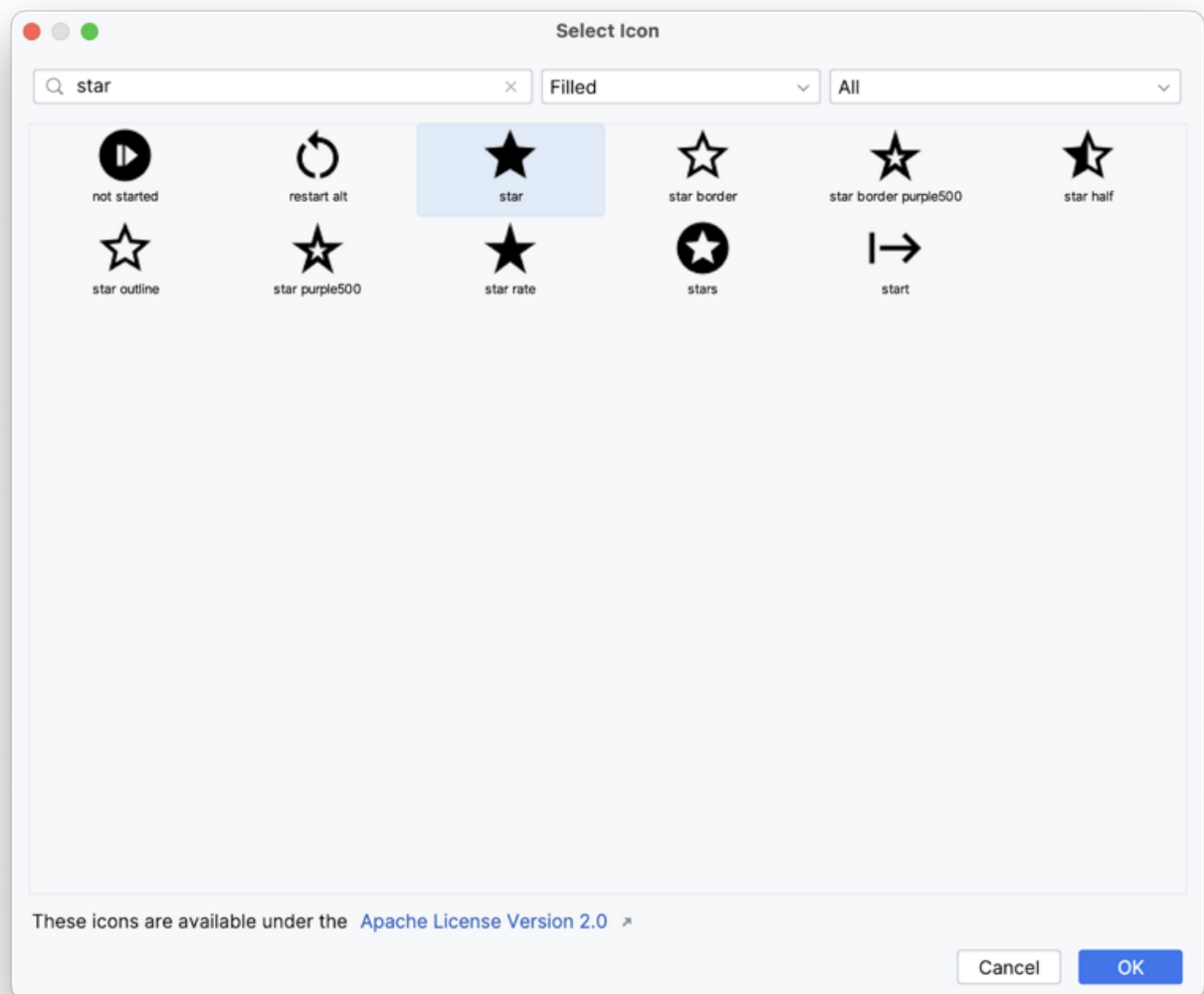
Для создания рисунка звезды в Compose необходимо создать векторный актив звезды.

На панели проекта щелкните правой кнопкой мыши на drawable > New > Vector Asset.

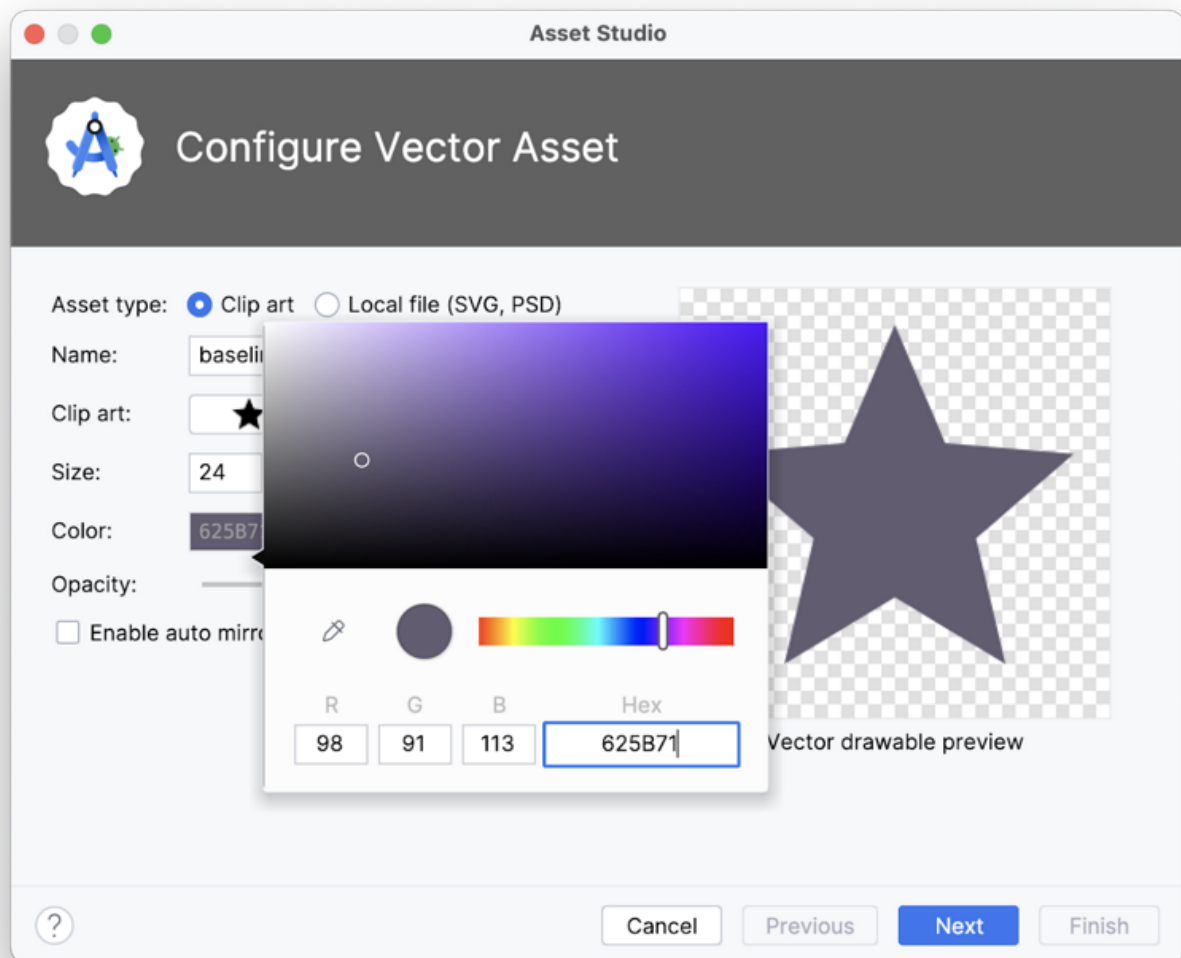


В диалоговом окне Asset Studio найдите значок звезды. Выберите заполненный значок звезды.





Измените значение цвета звезды на 625B71.



Нажмите кнопку Далее > Завершить. Обратите внимание, что в папке res/drawable появилась отрисовка.



Добавьте компонент предварительного просмотра, чтобы просмотреть компонент JuiceDetails.

```
@Preview
@Composable
fun PreviewJuiceDetails() {
    JuiceDetails(Juice(1, "Sweet Beet", "Apple, carrot, beet, and lemon", "Red",
```

```
4))  
}
```

PreviewJuiceDetails

Sweet Beet

Apple, carrot, beet, and lemon



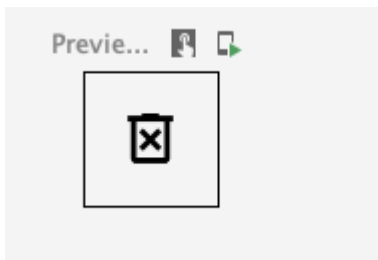
Создайте композитную кнопку удаления В файле `JuiceListAdapter.kt` добавьте еще одну композитную функцию `DeleteButton()`, которая принимает лямбду функции обратного вызова и модификатор. Установите лямбду в аргумент `onClick` и передайте `Icon()`, как показано в следующем коде:

```
import androidx.compose.ui.res.painterResource  
import androidx.compose.material3.Icon  
import androidx.compose.material3.IconButton  
  
@Composable  
fun DeleteButton(onDelete: () -> Unit, modifier: Modifier = Modifier) {  
    IconButton(  
        onClick = { onDelete() },  
        modifier = modifier  
    ) {  
        Icon(  
            painter = painterResource(R.drawable.ic_delete),  
            contentDescription = stringResource(R.string.delete)  
        )  
    }  
}
```

Добавьте функцию предварительного просмотра кнопки удаления.

```
@Preview  
@Composable
```

```
fun PreviewDeleteIcon() {
    DeleteButton({})
}
```



5. Реализация функции ListItem

Теперь, когда у вас есть все необходимые компоненты для отображения элементов списка, вы можете расположить их в макете. Обратите внимание на функцию `ListItem()`, которую вы определили в предыдущем шаге.

```
@Composable
fun ListItem(
    input: Juice,
    onEdit: (Juice) -> Unit,
    onDelete: (Juice) -> Unit,
    modifier: Modifier = Modifier
) {
}
```

В файле `JuiceListAdapter.kt` выполните следующие шаги для реализации функции `ListItem()`.

Добавьте макет `Row` внутри лямбды `Mdc3Theme {}`.

```
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import com.google.accompanist.themeadapter.material3.Mdc3Theme

Mdc3Theme {
    Row(
        modifier = modifier,
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
    }
}
```

Внутри лямбды `Row` вызовите три композита `JuiceIcon`, `JuiceDetails` и `DeleteButton`, которые вы создали в качестве дочерних элементов.

```
JuiceIcon(input.color)
JuiceDetails(input, Modifier.weight(1f))
DeleteButton({})
```

Передача `Modifier.weight(1f)` в составной элемент `JuiceDetails()` гарантирует, что детали сока займут горизонтальное пространство, оставшееся после измерения невзвешенных дочерних элементов.

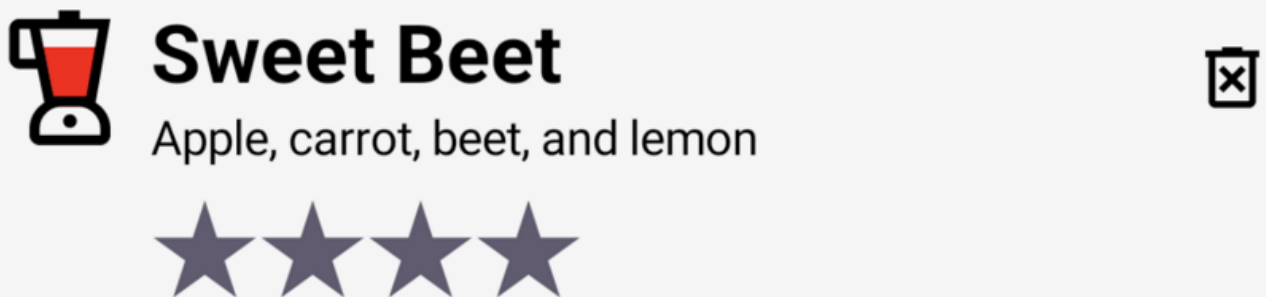
Передайте лямбду `onDelete(input)` и модификатор с выравниванием сверху в качестве параметров в составную кнопку `DeleteButton`.

```
DeleteButton(
    onDelete = {
        onDelete(input)
    },
    modifier = Modifier.align(Alignment.Top)
)
```

Напишите функцию предварительного просмотра, чтобы просмотреть составной `ListItem`.

```
@Preview
@Composable
fun PreviewListItem() {
    ListItem(Juice(1, "Sweet Beet", "Apple, carrot, beet, and lemon", "Red", 4),
    {})
}
```

PreviewListItem



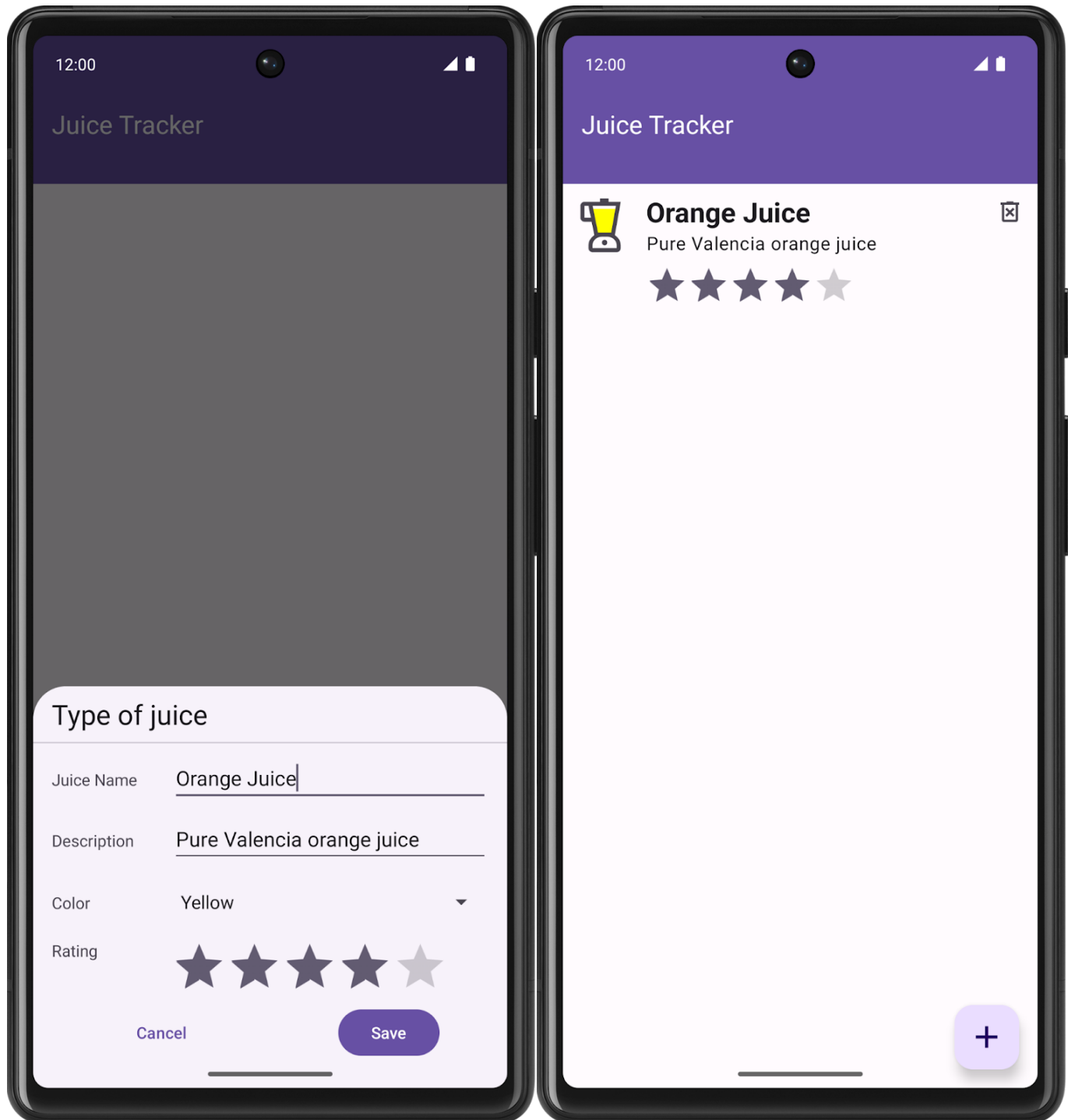
Привяжите составной `ListItem` к держателю представления. Вызовите `onEdit(input)` внутри лямбда-функции `clickable()`, чтобы открыть диалог редактирования при нажатии на элемент списка. В классе `JuiceViewHolder` внутри функции `bind()` необходимо разместить композитный элемент. Для этого используется `ComposeView` - представление Android, которое может размещать содержимое Compose UI с помощью своего метода `setContent`.

```
fun bind(input: Juice) {
    composeView.setContent {
```



```
        ListItem(
            input,
            onDelete,
            modifier = Modifier
                .fillMaxWidth()
                .clickable {
                    onEdit(input)
                }
            .padding(vertical = 8.dp, horizontal = 16.dp),
        )
    }
}
```

Запустите приложение. Добавьте свой любимый сок. Обратите внимание на блестящий элемент списка «Составить».



Поздравляем! Вы только что создали свое первое приложение для взаимодействия с Compose, которое использует элементы Compose в приложении, основанном на представлении.

6. Получение кода решения

Чтобы загрузить код готового codelab, вы можете использовать эти git-команды:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-juice-tracker.git
$ cd basic-android-kotlin-compose-training-juice-tracker
$ git checkout views-with-compose
```

